eedi-mining-misconceptions-in-mathematics

12th Place Solution

Rank	12
Team	chizhu
Author	chizhu
Topic ID	551429
Version	Original

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551429
Retrieved with Kaggle CLI on 2026-10-21.

I would like to express my gratitude to the Kaggle platform and Eedi for hosting this competition. Throughout this event, I have learned a great deal about recall and ranking techniques, especially on how to leverage large models for embedding contrastive learning and reranking. Ultimately, I achieved my solo gold medal, bringing me closer to becoming a Grand Master.

Overview

The primary strategy involved a Retriever combined with a reranker.

Synthetic Data Generation

Using the Chain of Thought (CoT) approach, I generated four batches of synthetic data, totaling over 10,000 external data points, based on misconceptions. The prompt was as follows:

```
You are an intelligent math problem-solving expert.
I will provide you with a misconception-related error reason. Based on this error, generate a math
single-choice question along with four options, where only one is correct, and the other three are
distractors.
The error reason provided corresponds to one of the incorrect options among the three distractors.
Below are several examples:
Example 1:
Input: Confuses the order of operations, believes addition comes before multiplication
Output: [question]ConstructName:Use the order of operations to carry out calculations involving
powersSubjectName:BIDMASQuestionDesc:[3 times 2+4-5]Where do the brackets need to go to make the
answer equal ( 13 ) ?[Candidate answers]A.( 3 times(2+4)-5 )B.( 3 times 2+(4-5) )C.( 3
times(2+4-5) )D.Does not need brackets[Correct Answer]( 3 times(2+4)-5 )[Misconcepte Incorrect
answer]Does not need brackets

Example 2:
Input: Does not know that to factorise a quadratic expression, to find two numbers that add to give
the coefficient of the x term, and multiply to give the non-variable term
Output: [question]ConstructName:Simplify an algebraic fraction by factorising the
numeratorSubjectName:Simplifying Algebraic FractionsQuestionDesc:Simplify the following, if possible:
( frac{m^{2}+2 m-3}{m-3} )[Candidate answers]A.( m+1 )B.( m+2 )C.( m-1 )D.Does not simplify[Correct
Answer]Does not simplify[Misconcepte Incorrect answer]( m+1 )

Example 3:
Input: Thinks that when you cancel identical terms from the numerator and denominator, they just
disappear
Output: [question]ConstructName:Simplify an algebraic fraction by factorising the
numeratorSubjectName:Simplifying Algebraic FractionsQuestionDesc:Simplify the following, if possible:
( frac{m^{2}+2 m-3}{m-3} )[Candidate answers]A.( m+1 )B.( m+2 )C.( m-1 )D.Does not simplify[Correct
Answer]Does not simplify[Misconcepte Incorrect answer]( m+2 )

Example 4:
Input: Does not know that to factorise a quadratic expression, to find two numbers that add to give
the coefficient of the x term, and multiply to give the non-variable term
Output: [question]ConstructName:Simplify an algebraic fraction by factorising the
numeratorSubjectName:Simplifying Algebraic FractionsQuestionDesc:Simplify the following, if possible:
( frac{m^{2}+2 m-3}{m-3} )[Candidate answers]A.( m+1 )B.( m+2 )C.( m-1 )D.Does not simplify[Correct
Answer]Does not simplify[Misconcepte Incorrect answer]( m-1 )
Please generate similar questions and options based on the cases above.
You first need to think about the reasoning logic according to the chain of thought, then provide the
specific case.
The output format is:
<thought>Insert your thoughts on the error reason here</thought>
<output>Insert your question and options here</output>
```

The external data helped improve recall from 0.430 to 0.470 (single fold).

Retriever

The overall approach involved initially using the SFR model to retrieve 100 candidates, followed by training contrastive learning with the Qwen model. The ensemble of three models was achieved by concatenating embeddings: `emb=concat(emb1,emb2,emb3)`.

- Inferred potential errors using Qwen2.5-32b-instruct-awq, combined with the original query to train Qwen2.5-14B-instruct. The 5-fold LoRA weights were averaged to obtain `emb1`.
- Trained Qwen2.5-14B-instruct with the original query, and averaged the 5-fold LoRA weights to get `emb2`.
- Used the open-source model from notebook for `emb3`.
- Public score: 0.526, Private score: 0.485

Reranker

For the top 50 candidates retrieved by the Retriever (results obtained from 5 folds oof), 25 were randomly selected each time for 25-class prediction. The final score was achieved by ensemble weighting the probabilities of three models:

- Public score: 0.597, Private score: 0.564
- Qwen2.5-32b-instruct-awq origin query + vllm infer: Averaged 5-fold LoRA weights, Public: 0.570
- Qwen2.5-14b-instruct origin query: Averaged 5-fold LoRA weights, Public: 0.567
- Qwen2.5-14b-instruct query + reason: Averaged 5-fold LoRA weights, Public: 0.558

What Didn't Work

- Qwen2.5-coder
- LGB/CatBoost for oof post reranker
- Reranker query + misconception last_pool => 0/1